



PROJECT REPORT

빵디즈 프로젝트

중간 보고 빵빠레

주병규(팀장) | 조형욱 | 노여진 | 김재성 | 손하영

목차



프로젝트 소개

주제, 문제 정의, 목표 및 핵심 기능, 개발 일정, 설계도 등 프로젝트의 전반적인 개요를 소개합니다.



협업 도구 & 인프라

MSA 아키텍처, 브로커 선택, CI/CD 파이프라인 및 모노레포 구조에 대해 설명합니다.



서비스별 설계

채팅, 인증·멤버, 주문, 결제, 알림 등 서비스별 도메인 설계와 핵심 기술 적용 사례를 소개합니다.



SECTION 01

프로젝트 소개

주제, 문제 정의, 목표 및 핵심 기능,
개발 일정, 설계도 등 프로젝트의 전반적인 개요를 소개합니다.

주제 및 목표

참여형 크라우드 펀딩 플랫폼

빵디즈는 사용자 간의 자유로운 펀딩을 지원하며, 펀딩 기간 동안 열리는 **실시간 공개방**을 통해 상품에 대한 토론과 소통을 이어가는 혁신적인 플랫폼입니다.



문제 정의

소통의 부재와 실패한 후원

기존 대형 펀딩 플랫폼은 후원자와 창작자, 또는 후원자 간의 직접적인 소통 수단이 매우 부족합니다.



시 스토리 요약 by WAI (Beta)

해리포터 공식 DIY 일렉션
위너브라더스 공식 라이선스 정품, 6종 스페셜 에디션

직접 조립·색칠하는 마법 체험
북북 미니어처, 원목 우드 퍼즐, 넘버즈 페인팅 포함

와디즈 단독 한정 판매 혜택
와디즈에서만 만나는 최초 공개 및 한정수량 판매

국내단독
국내에서는 오직 와디즈에서만 단독으로 선보이는 제품입니다.

프리오더
한정과 기존 출시된 제품 서비스 콘텐츠를 특별구성, 단독유용, 한정판으로 혜택을 제공합니다.

스토어 새 소식 5 커뮤니티 539 서포터 940 환불·정책

새 소식 달성률
실시간 펀딩 1위 달성! 감사합니다

2026 와디즈 슈퍼메이커
높은 만족도와 성과를 얻은 우수 메이커의 프로젝트예요.

도시산간에 해당하는 시포터(님)은 배송 가능 여부를 반드시 메이커에게 문의 후 펀딩에 참여해 주세요.

프로젝트 스토리

페이지를 넘겨서든,
스크린을 통해서든,
언젠가 당신이 다시 돌아왔을 때,

캐릭터 굿즈 랭킹 1위
오 에어로코리아 (주) 슈퍼메이커

[해리포터 30주년] 국내 단독, 공식 DIY 굿즈 스페셜 에디션 6종

13,277 % 달성 9월
66,386,200원 달성 940명 참여

지금 SNS에서 핫한 프로젝트

혜택 서포터클럽 3개월 무료 혜택 받기
5만 원 이상 결제/아니 첫 결제 최대 3천 원 혜택

공유 5천+ 485 예약 구매하기

485 명이 지지서명했어요
메이커의 멋진 아이디어를 세상에 알려주세요.
지지서명을 공유하시면 포인트를 드립니다.

지지서명 하기

슈퍼메이커
에어로코리아 (주)
리워드 정보
문의하기
더보기

프로젝트에 문제가 있나요? 신고하기

리워드 선택
진행 기간 2026.07.14 ~ 2026.07.24

슈퍼 메이커 54개 남음
[한정수량 | BEST] 해리포터 공식 북북 1+1 세트 (송강창,연희정)
39% 144,800원
238,000원

슈퍼 메이커 183개 남음
[한정수량] 해리포터 공식 3종 세트 (북북, 퍼즐, 페인팅)
40% 123,800원
207,800원

슈퍼 메이커 153개 남음
[한정수량] 해리포터 공식 3종 세트 1+1 (북북, 퍼즐, 페인팅)
43% 238,000원
418,600원

슈퍼 메이커 189개 남음
[한정수량] 해리포터 공식 2종 세트 (북북, 퍼즐)
39% 109,000원
178,000원

해결 방안



실시간 소통을 통한 검증

빵디즈는 펀딩별 **공개 채팅방**을 도입하여 제품에 대한 심도 있는 토의를 가능하게 합니다.

이를 통해 후원자가 "나에게 정말 필요한 상품인가"를 객관적으로 판단할 기준점을 스스로 세우도록 돕습니다. 결과적으로 맹목적인 충동 후원을 방지하고, 펀딩의 질적 향상과 만족도를 높이는 경험을 제공합니다.

목표의 핵심 기능

커뮤니티형 펀딩

단순한 구매를 넘어, 관심사가 같은 사용자들이 모여 자유롭게 의견을 나누고 상품의 가치를 함께 발전시킵니다.

스마트 펀딩 시스템

누구나 쉽게 참여하고 진행 상황을 투명하게 확인할 수 있는 안정적인 펀딩 환경을 제공합니다.

실시간 알림 서비스

펀딩 성공 여부, 공개방의 주요 대화, 새로운 소식 등 놓치기 쉬운 핵심 정보를 실시간으로 신속하게 전달합니다.

자체 월렛 기반 결제 시스템

사이트 전용 월렛을 통해 결제하며, 펀딩 성공/실패 상황에 따라 자동 환불이 이루어져 안전한 자금 흐름을 보장합니다.



SECTION 02

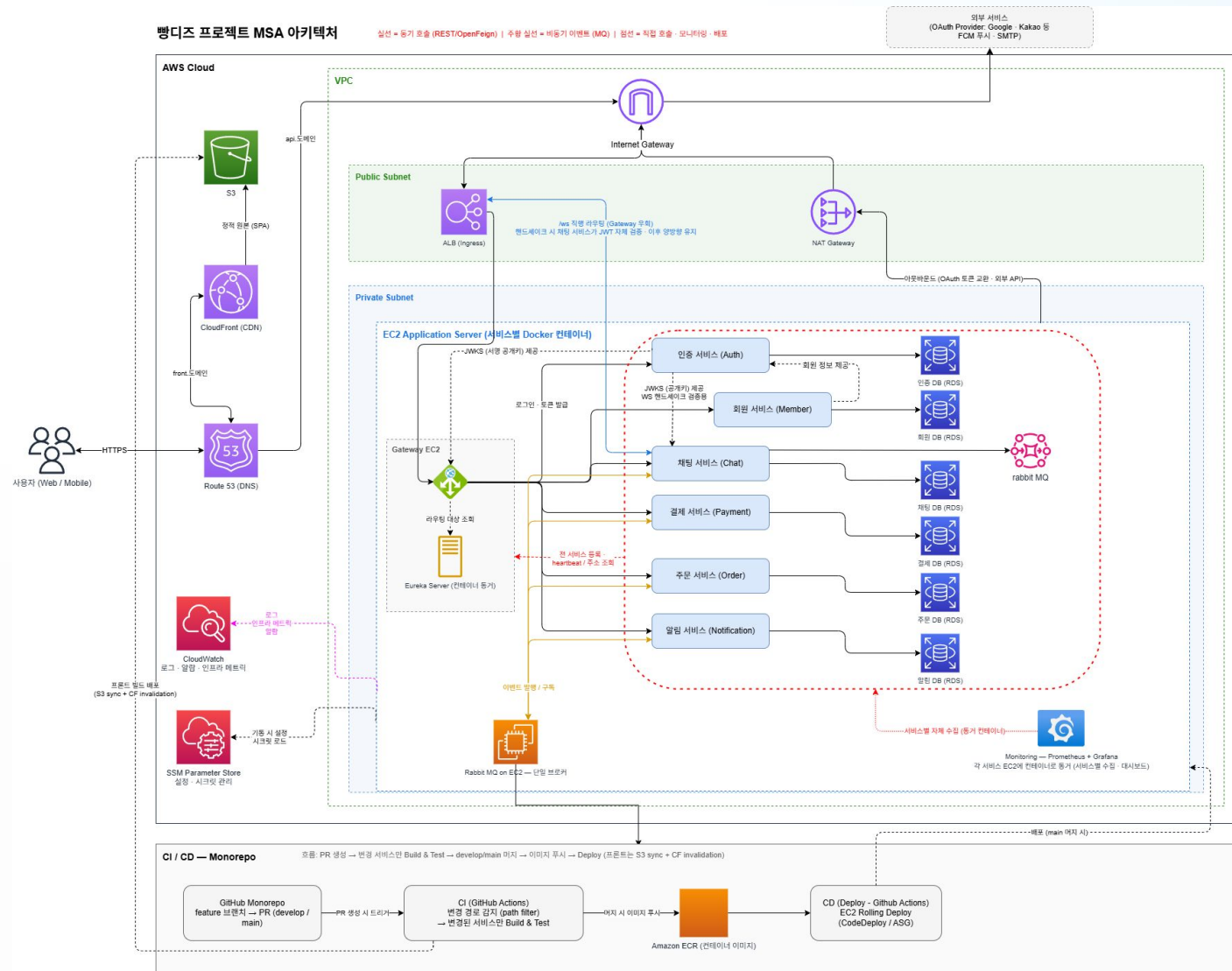
협업 도구 & 인프라

MSA 아키텍처, 브로커 선택(RabbitMQ),
CI/CD 파이프라인 및 모노레포 구조에 대해 설명합니다.

기술 스택 및 아키텍처

분류	상세 기술 스택
아키텍처 & 언어	MSA (Microservices Architecture), Java 25
프레임워크	Spring Boot 4.1.0, Spring Cloud 2025.1.2 (Oakwood)
인증 및 실시간	Spring Security, JWT, WebSocket, STOMP
MSA 통신 & 라우팅	RabbitMQ:4, Spring Cloud Gateway, Eureka (Service Discovery)
테스트 & 인프라	JUnit 5, Mockito, Testcontainers, k6, Docker, Docker Compose

기술 스택 및 아키텍처



Event Broker 선택: RabbitMQ

RabbitMQ 도입 배경

빵디즈의 MSA 환경은 대규모 스트리밍보다 간헐적인 Event Publish가 주를 이룹니다.

- **리소스 효율성:** Kafka의 Pull 방식(지속적 poll)과 달리, Push 방식은 메시지가 없을 때 TCP 연결만 유지.
- **가벼운 구동:** 컨테이너 기동 시 관리 UI 포함 약 200MB 메모리로 3초 만에 실행 (Kafka는 최소 1GB 요구).

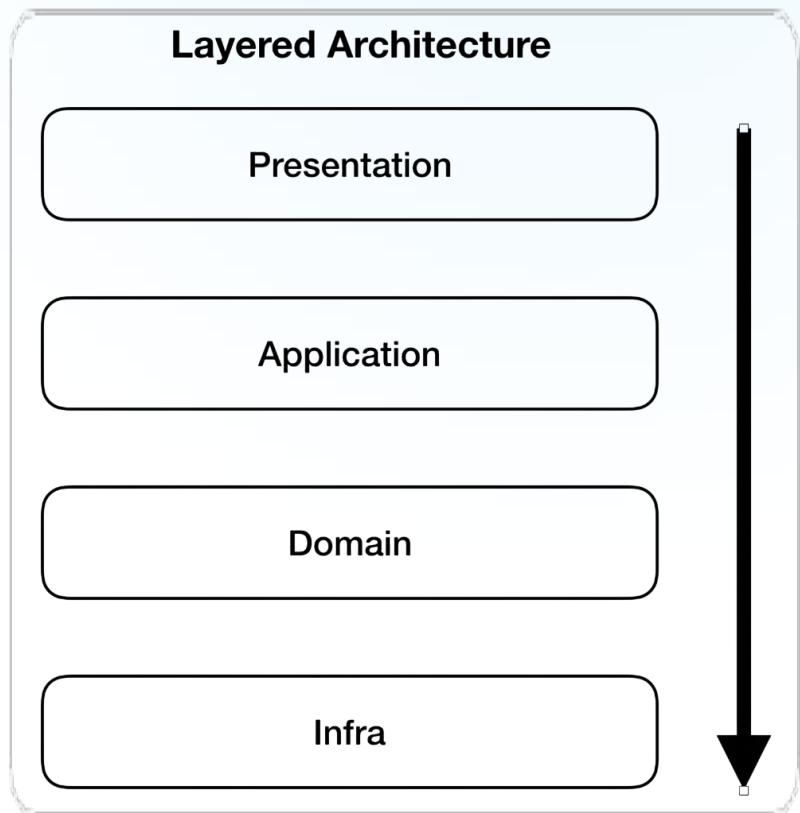
운영 및 복구 편의성

인프라적 해결을 통한 개발 및 운영 난이도 감소를 목표로 했습니다.

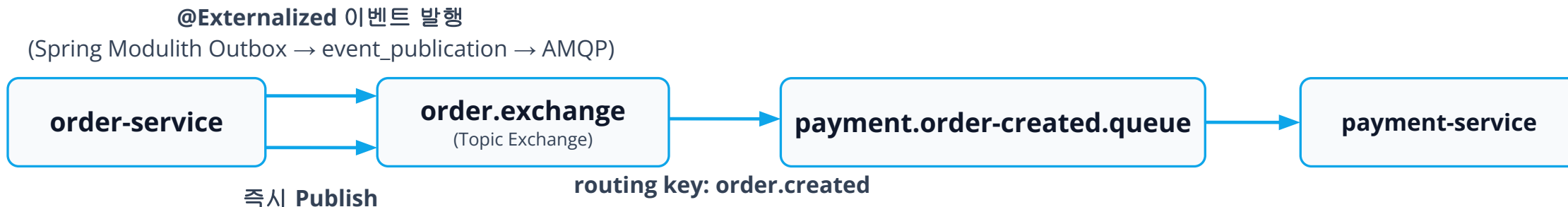
- **자동 DLQ 처리:** 큐 선언 시 x-dead-letter-exchange 속성 한 줄로 실패 메시지 자동 이동. (Kafka는 직접 복잡한 핸들링 필요)
- **장애 복구:** 브로커 다운 후 복구 시, 미처리 메시지 자동 환원(requeue) 및 단순 재연결 지원. (Kafka의 복잡한 Rebalancing 회피)

디렉터리 구조 (Monorepo)

- 📁 bds_backend/
 - ├── 📁 .github/ // CI/CD 워크플로우, 이슈 템플릿
 - ├── 📁 docs/ // API 스펙, 프로젝트 가이드
 - ├── 📁 infra/ // RabbitMQ 로컬/dev 설정
 - ├── 📁 libs/ // 서비스 간 공통 이벤트 타입
 - ├── 📁 modules/ // 공통 유틸리티, 메시징 설정(RabbitMQ 공통 설정)
 - ├── 📁 platform/ // Config / Discovery 서버
 - └── 📁 services/
 - ├── auth-service
 - ├── chat-service
 - ├── member-service
 - └── ...기타 서비스



서비스 간 통신 구조



Topic Exchange

서비스 간 메시지는 RabbitMQ Topic Exchange를 통해 비동기로 전달되거나 http request를 통해 동기적으로 전달됩니다.

Outbox Pattern

유실 불가 이벤트는 Spring Modulith Outbox 패턴을 적용하고, 유실 허용 이벤트는 Outbox DB 저장 없이 바로 publish합니다.

Quorum & DLQ

모든 큐는 Quorum Queue 기반이며, 재시도 5회 초과 시 DLQ(Dead Letter Queue)로 이동합니다.

서비스 간 통신 방식

연결 목적에 따라 비동기(RabbitMQ)와 동기(HTTP/OpenFeign) 통신을 구분해 사용합니다.

ASYNC

비동기 통신

주문 ↔ 결제

RabbitMQ(Broker)를 사용하여 비동기적으로 통신합니다.

편딩 ↔ 채팅

RabbitMQ(Broker)를 사용하여 비동기적으로 통신합니다.

채팅 · 주문 · 편딩 ↔ 알림

RabbitMQ(Broker)를 사용하여 비동기적으로 통신합니다.

SYNC

동기 통신

인증 ↔ 채팅 (Spring Cloud Gateway)

HTTP Request를 사용하여 동기적으로 통신합니다.

인증 ↔ 멤버

OpenFeign을 사용하여 동기적으로 통신합니다.

CI 구조 (Multi-module)



서비스 독립 파이프라인 각 서비스마다 별도의 워크플로우 파일을 구성하여, 변경이 발생한 서비스만 선별적으로 빌드 및 테스트를 실행합니다.



최적화 기법 적용 Sparse Checkout으로 해당 서비스와 의존 모듈만 체크아웃하여 시간을 단축하고, Gradle 의존성 캐시를 활용해 매 빌드 재다운로드를 방지합니다.



Matrix 빌드 전략 변경 감지(detect-changes) Job을 통해 git diff로 변경된 서비스를 추출하고, 해당 서비스 목록을 기반으로 병렬 빌드 및 테스트를 수행합니다.



Jacoco 커버리지 auth, chat, member, payment 등 각 서비스 단위로 커버리지 리포트를 수집하여 품질을 관리합니다.

개발 일정



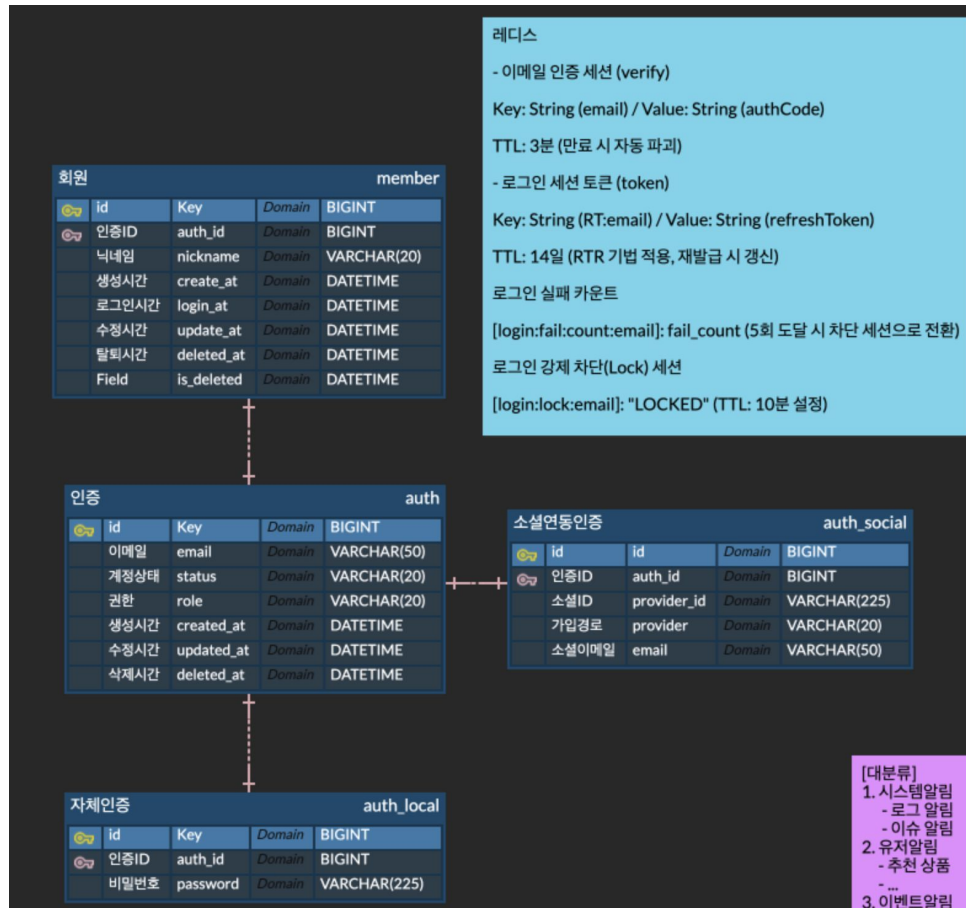


SERVICE 01

인증 · 멤버

회원 인증/인가 구조와 RSA·JWKS, 게이트웨이 중심 JWT 검증 아키텍처를
다룹니다.

ERD



인증 / 회원 분리

- 프로필 데이터를 담은 회원과 비밀번호·계정상태를 담은 인증을 분리했습니다.

레디스 활용

- 이메일 인증코드, refresh 토큰 등 일정 시간 뒤 자동 소멸하는 데이터를 RDB 대신 Redis TTL로 관리합니다.

EndPoint

Authentication 도메인

엔드포인트 목록

method	path	auth required	설명
POST	/api/auths/mail	N	이메일 인증 번호 발송(1)
POST	/api/auths/mailCheck	N	이메일 인증 번호 검증(2)
POST	/api/auths/login	N	로그인 및 토큰 발급(3)
POST	/api/auths/token/refresh	N	refresh 토큰 재발급(5)
POST	/api/auths/password/verify	N	비밀번호 변경 권한 획득(6)
GET	/api/auths/logout	O	로그아웃(7)

Member 도메인

엔드포인트 목록

method	path	auth required	설명
POST	/api/members/signup	N	플랫폼 자체 회원가입(1)
POST	/api/members/social	N	소셜 회원가입 및 로그인(2)
GET	/api/members/info	O	내 정보(이메일, 닉네임) 조회(6)
PATCH	/api/members/info	O	내 정보(닉네임) 수정(7)
DELETE	/api/members/delete	O	회원 탈퇴(10)
PATCH	/api/members/role	O	회원 권한 변경(서포터, 메이커)(11)

Jacoco Coverage

auth-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.bds.auth.domain.entity		96%		92%
com.bds.auth.application		99%		94%
com.bds.auth.infrastructure.security		100%		n/a
Total	6 of 668	99%	2 of 32	93%

gateway-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.dbs.gateway.filter		98%		100%
com.dbs.gateway.config		100%		n/a
com.dbs.gateway.security		100%		n/a
Total	2 of 176	98%	0 of 4	100%

member-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.bds.member.application		100%		87%
com.bds.member.domain.entity		100%		90%
Total	0 of 201	100%	3 of 26	88%

entity, service 위주로 테스트 코드 작성

보안 강화: 비대칭키 (RSA)와 JWKS

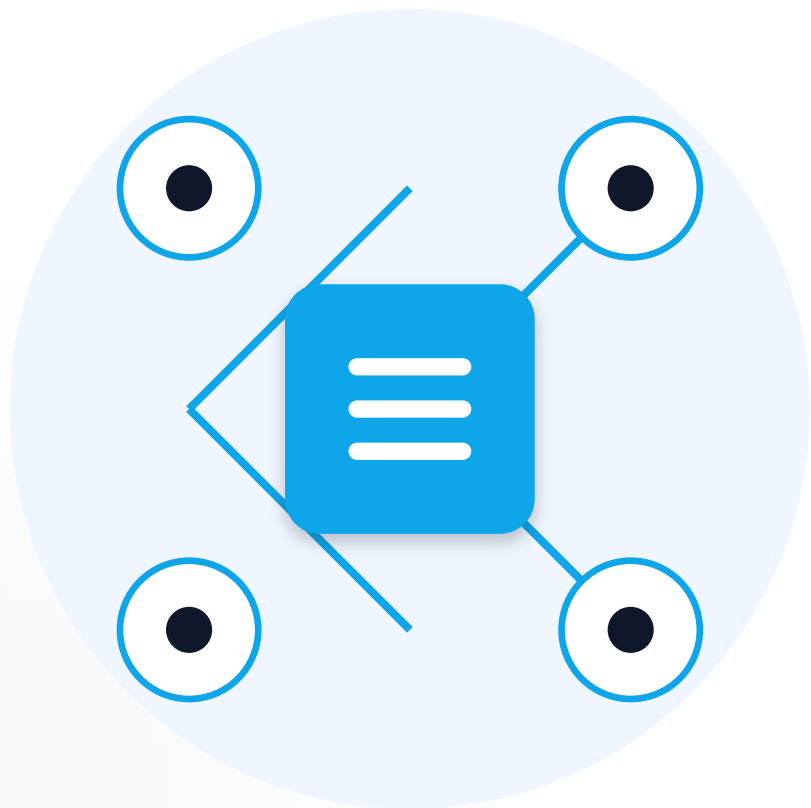


대칭키의 한계 극복

단일 시크릿(대칭키) 사용 시 특정 서비스 유출만으로도 전체 토큰 위조가 가능한 보안 취약점이 있었습니다.

해결 방안: RSA 비대칭키를 도입하여 auth-service만이 개인키로 서명합니다. 다른 도메인들은 /oauth2/jwks 엔드포인트를 통해 공개키만 제공받아 검증을 수행합니다. 이를 통해 키 로테이션 시에도 타 서비스의 재배포 없이 실시간 동기화가 보장됩니다.

게이트웨이 중심 JWT 검증

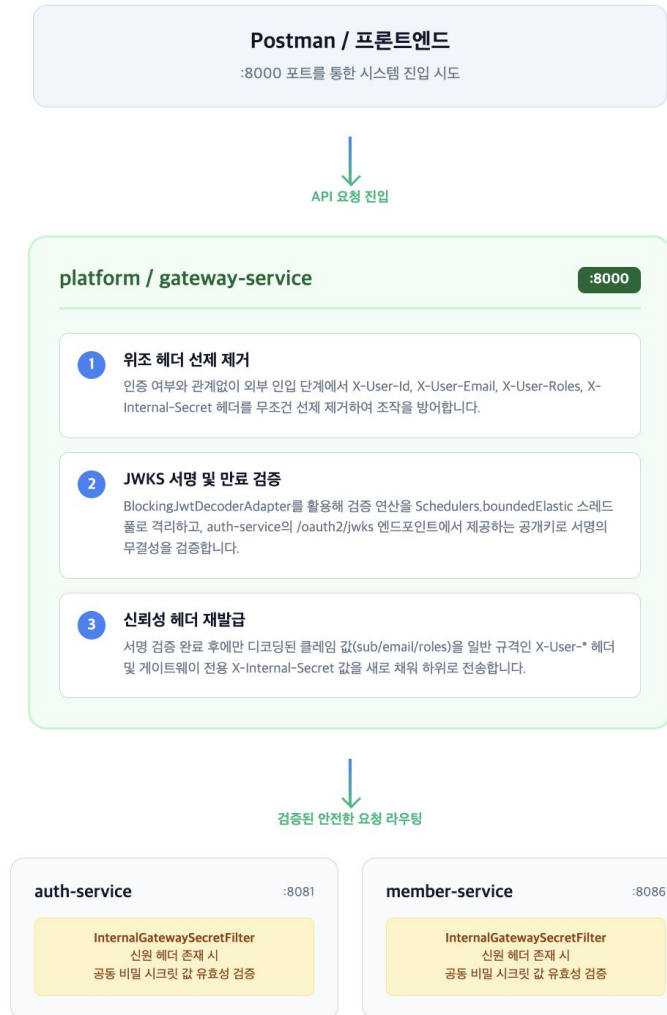


개별 검증 로직의 한계

서비스가 6개면 각 서비스가 JWT를 개별적으로 검증해야 해, 시크릿 키가 여러 곳에 흩어지고 검증 로직이 중복되는 문제가 있었습니다.

해결 방안: 게이트웨이를 앞단에 세워 인증 검증을 전담시킵니다. 클라이언트는 게이트웨이만 바라보고, 게이트웨이가 경로를 보고 알맞은 서비스로 라우팅합니다.

헤더 매핑, 위조 방지



1 게이트웨이 레벨

- 클라이언트가 임의로 조작해 보낸 헤더를 강제로 제거합니다.
- 서명 검증에 성공했을 때만 sub / email / roles와 비밀값을 헤더에 넣어줍니다.

2 서비스 레벨

- 각 서비스 진입 시 게이트웨이가 넣어준 비밀값의 정합성을 검증합니다.
- 게이트웨이를 거치지 않고 서비스로 직접 들어오는 요청은 차단합니다.

회원 생명주기



- 회원가입, 탈퇴 생명주기 처리를 위해 설계했습니다.
- 회원 정보의 상태 변화가 일어나는 시점에는 OpenFeign을 활용한 실시간 API 연동을 사용합니다.



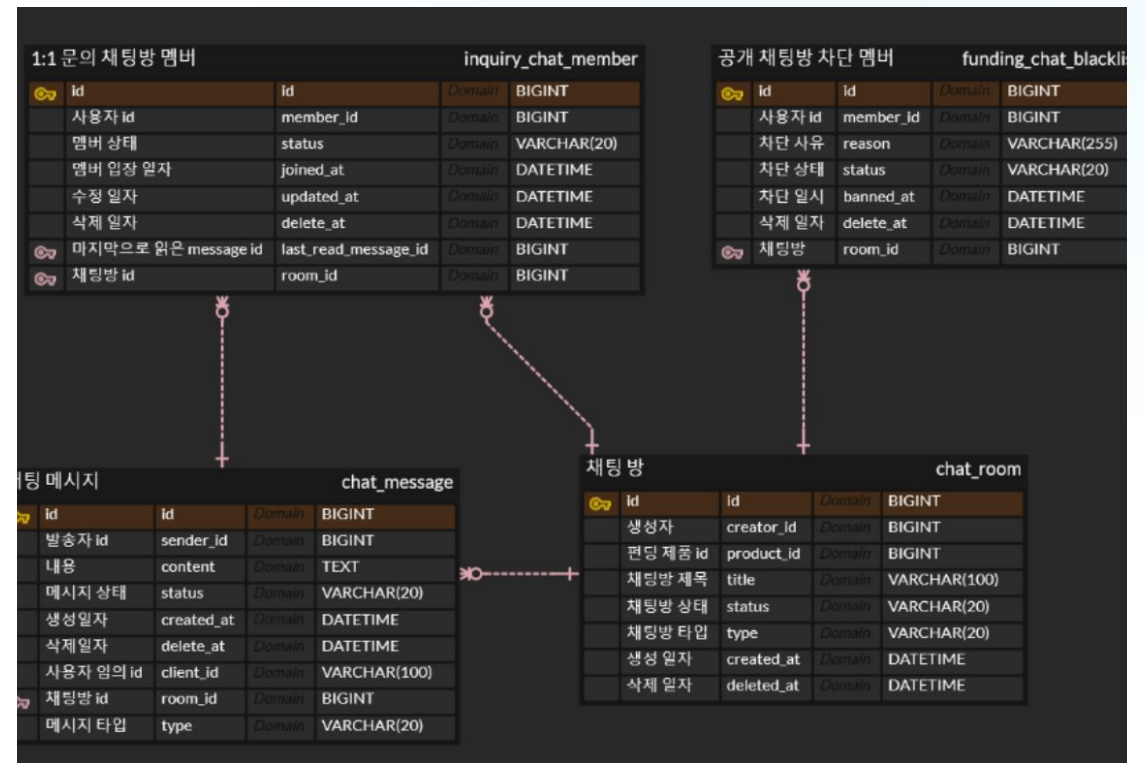
SERVICE 02

채팅

WebSocket(STOMP) 기반 실시간 채팅과 STOMP 인증, 세션·토큰
생명주기를 다룹니다.

ERD

- **채팅방 유형:** 공개 펀딩 채팅방과 1:1 문의 채팅방으로 분리되어 운영됩니다.
- **멤버 관리:** 1:1 문의방은 명시적인 멤버 정보를 저장하며, 공개방은 별도 멤버 정보 없이 자유로운 참여를 보장합니다.
- **관리 기능:** 악성 유저 차단을 위한 Blacklist(Ban) 기능은 공개 채팅방에 한정하여 지원합니다.



EndPoint

method	path	auth required	설명
POST	/api/chat/inquiry	O	1:1 문의 채팅방 생성
GET	/api/chat/inquiry	O	내 참여 문의 채팅방 목록 조회
GET	/api/chat/inquiry/{roomId}	O	1:1 문의 채팅방 상세 조회
PATCH	/api/chat/rooms/{roomId}/close	O	공개 채팅방 삭제
POST	/internal/chat/funding/{productId}	X	펀딩 제품 생성시 공개 채팅방 자동 생성 (시스템)
GET	/api/chat/funding/{productId}	X	공개 채팅방 조회
POST	/api/chat/funding/{roomId}/ban	O	공개 채팅방 사용자 BAN
GET	/api/chat/rooms/messages	O	채팅 이력 조회
GET	/api/chat/inquiry/{roomId}/messages	O	1:1 문의 채팅방 메시지 조회
GET	/api/chat/funding/{roomId}/messages	O	공개 채팅방 메시지 조회
DELETE	/api/chat/messages/{messageId}	O	메시지 삭제(soft delete)
WS	/ws/chat	X	Websocket 연결
SUB	/topic/rooms/{roomId}	O/X	채팅방 구독
UNSUB	subscription-id	O/X	채팅방 구독 취소
PUB	/app/chat.send	O	메시지 전송
PUB	/app/chat.read	O	읽음 상태 갱신
PUB	/app/chat.typing	O	타이핑 인디케이터

Jacoco Coverage

chat-service

[Sessions](#)

chat-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.bds.chat.infrastructure.session		84%		64%	36	93	36	193	5	47	0	10
com.bds.chat.domain.shared		59%		52%	24	54	17	66	7	30	0	6
com.bds.chat.application.chatRoom.dto		74%		25%	6	19	14	49	4	17	2	10
com.bds.chat.infrastructure.persistence.message		80%		73%	10	32	15	75	4	19	0	2
com.bds.chat.presentation		71%		60%	9	19	15	47	3	10	0	1
com.bds.chat.infrastructure.security		90%		55%	9	25	6	63	1	16	0	3
com.bds.chat.application.chatRoom.service		95%		82%	5	50	2	124	0	36	0	2
com.bds.chat.presentation.stomp		89%		83%	1	13	5	46	0	10	0	7
com.bds.chat.application.message.service		95%		81%	8	43	4	82	1	21	0	1
com.bds.chat.domain.chatRoom		86%		62%	3	13	6	35	1	9	0	3
com.bds.chat.common		83%		n/a	3	12	8	27	3	12	1	5
com.bds.chat.presentation.stomp.dto		75%		n/a	1	4	1	5	1	4	1	2
com.bds.chat.infrastructure.config		96%		50%	2	23	2	74	1	22	0	6
com.bds.chat.infrastructure.persistence.member		96%		100%	0	19	2	55	0	17	0	2
com.bds.chat		37%		n/a	1	2	2	3	1	2	0	1
com.bds.chat.domain.member		99%		91%	2	27	0	54	0	15	0	3
com.bds.chat.application.message.dto		98%		50%	1	7	0	12	0	6	0	4
com.bds.chat.application.blackList.service		100%		100%	0	11	0	29	0	5	0	1
com.bds.chat.infrastructure.persistence.chatroom		100%		100%	0	12	0	38	0	9	0	2
com.bds.chat.domain.message		100%		100%	0	13	0	30	0	10	0	4
com.bds.chat.infrastructure.persistence.blacklist		100%		100%	0	8	0	27	0	6	0	2
com.bds.chat.domain.blackList		100%		100%	0	8	0	23	0	6	0	2
com.bds.chat.presentation.chatRoom		100%		n/a	0	7	0	13	0	7	0	2
com.bds.chat.application.member.service		100%		n/a	0	4	0	12	0	4	0	1
com.bds.chat.presentation.message		100%		n/a	0	4	0	4	0	4	0	1
com.bds.chat.application.member.dto		100%		n/a	0	2	0	6	0	2	0	1
com.bds.chat.presentation.blackList		100%		n/a	0	2	0	4	0	2	0	1
com.bds.chat.application.blackList.dto		100%		n/a	0	2	0	2	0	2	0	2
Total	651 of 5,800	88%	100 of 351	71%	121	528	135	1,198	32	350	4	87

채팅 기술 스택



실시간 통신

WebSocket + STOMP으로 실시간
메시지 송수신



메시지 브로커

RabbitMQ 4 + STOMP
플러그인으로
외부 브로커 릴레이로 사용합니다.



데이터 베이스

PostgreSQL과 Spring Data JPA를
통해 영구 저장소를 구성하며,
Flyway로 스키마 버전을 체계적으로
관리합니다.

STOMP 인증: Gateway 우회 직접 인증

직접 인증 배경

일반 REST API는 Gateway가 JWT를 검증하지만, WebSocket은 핸드셰이크 이후 STOMP 프레임 단위로 동작해 Intercept가 어렵습니다.

따라서 **chat-service**가 **auth-service**의 JWKS 엔드포인트에서 공개키를 직접 조회하여 토큰 서명을 검증하는 구조를 채택했습니다.

CONNECT 흐름 및 익명 세션

- **인증 흐름:** STOMP CONNECT 시 Bearer 토큰 수신
→ JWKS 검증 → StompPrincipal 등록 → 세션 및 토큰 갱신 타이머 예약.
- **익명 접속:** 토큰이 없으면 익명 세션(최대 1,000 슬롯)으로 허용하여, 공개방 구독(Subscribe)은 가능하되 메시지 송신(Send)은 원천 차단합니다.

세션 레지스트리 (SessionContextRegistry)

WebSocketHandlerDecoratorFactory 구현체로, WebSocket 연결 수립·종료 이벤트에 자동으로 개입합니다.

세션 생명주기

- 연결 수립 → `contexts.put(sessionId, SessionContext)`
- CONNECT 인증 성공 → `attachAuth` (authState 저장 + 유저별 세션 등록)
- 토큰 갱신 성공 → `extendAuth` (새 `expiresAt`으로 authState 재생성)
- 세션 강제 종료 → `invalidateAuth` 후 소켓 종료
- 연결 해제 → `contexts` 제거, 익명이면 카운트 감소, 유저-세션 매핑 정리

관리 자료구조

- `contexts: ConcurrentHashMap<sessionId, SessionContext>`
- `SessionContext = { WebSocketSession(강제종료용), volatile AuthState(userId·roles·expiresAt·valid), volatile anonymous }`
- `sessionIdsByUser: ConcurrentHashMap<userId, Set<sessionId>>`
— 한 유저의 다중 탭/기기 세션 추적
- `anonymousCount: AtomicInteger` — 익명 슬롯 동시 수 관리

토큰 생명주기

STOMP는 연결이 장시간 유지되므로 JWT 만료 시 연결이 끊기는 문제가 생깁니다. TokenRenewalManager는 만료 전 클라이언트에 갱신을 요청하고, 응답이 없으면 세션을 능동적으로 종료합니다.

generation 카운터

- schedule() 호출마다 세션별 generation을 증가
- expire() 실행 시 저장된 generation과 현재 값이 다르면 오발동으로 판단해 취소
- → 연속 토큰 갱신 시 이전 killTask가 정상 세션을 끊는 것을 방지

BrokerAvailabilityListener

- STOMP relay(RabbitMQ) 연결 상태를 BrokerAvailabilityEvent로 감시
- BROKER-DOWN 시 에러 로그, 복구 시 INFO 로그 기록
- isAvailable() 헬스체크에 활용

갱신 · 종료 흐름

- 만료 전(기본 2분) renewalTask 실행 → 클라이언트에 TOKEN_RENEWAL_REQUIRED 푸시
- grace 1분 내 미갱신 시 killTask 실행 → 세션 강제 종료
- 갱신 성공 시 killTask 취소 후 재스케줄



SERVICE 03

주문

주문 도메인 설계와 Funding·Reward 처리, RabbitMQ 기반 이벤트 흐름을
다룹니다.

ERD



- 주문 서비스에 Funding & Reward 컨텍스트를 포함합니다.
- order_reward로 다대다 분리, 주문 시점 금액을 스냅샷으로 보관합니다.
- event_publication: Outbox 패턴을 적용해 이벤트 유실을 방지하며, 도메인 테이블과 독립적으로 관리합니다.

EndPoint

Order 도메인

method	path	auth	설명
POST	/api/orders/billing	O	주문서 생성 (정보 조회, DB X)
POST	/api/orders	O	주문 생성 (재고 차감 + 결제 호출)
GET	/api/orders	O	내 주문 목록 조회
GET	/api/orders/{orderId}	O	주문 상세 조회
PATCH	/api/orders/{orderId}/cancel	O	사용자 주문 취소

Funding 도메인

method	path	auth	설명
GET	/api/fundings	X	펀딩 목록 조회 (상태 필터링 가능)
GET	/api/fundings/{fundingId}	X	펀딩 상세 및 리워드 목록 조회
POST	/api/fundings	O	펀딩 생성 (리워드 동시 등록, 메이커 전용)

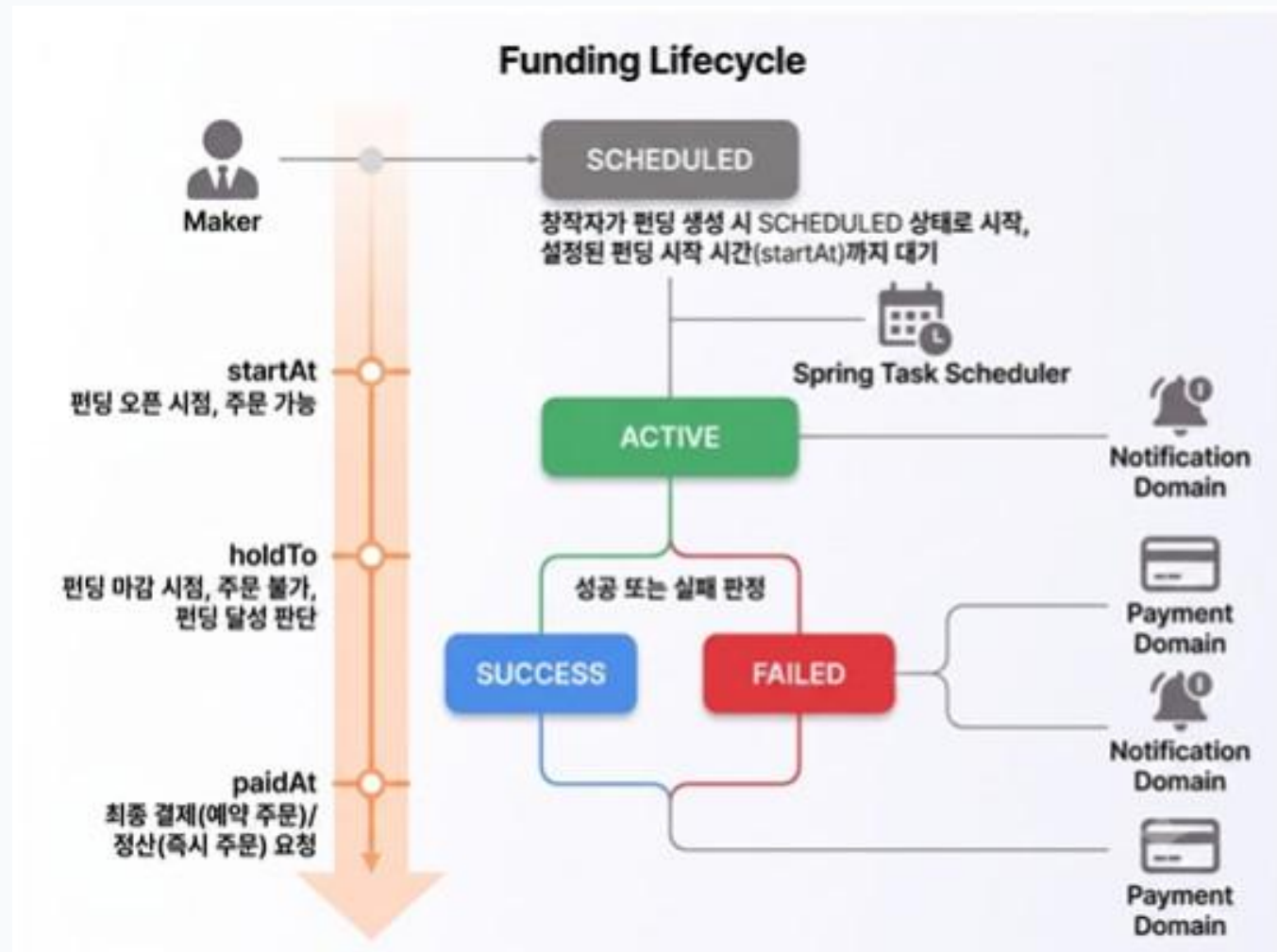
Jacoco Coverage

order-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.bds.order.infrastructure.scheduler		97%		100%	2	27	0	88	2	14	0	3
com.bds.order.application		98%		100%	1	47	3	132	1	32	0	3
com.bds.order.domain.order		99%		97%	1	31	0	41	0	12	0	3
com.bds.order.presentation.dto		100%		100%	0	25	0	83	0	24	0	15
com.bds.order.global.exception		100%		n/a	0	23	0	80	0	23	0	4
com.bds.order.infrastructure.order		100%		100%	0	14	0	52	0	12	0	5
com.bds.order.infrastructure.funding		100%		100%	0	16	0	65	0	15	0	3
com.bds.order.domain.funding		100%		100%	0	9	0	19	0	7	0	2
com.bds.order.presentation.controller		100%		n/a	0	8	0	15	0	8	0	2
com.bds.order.infrastructure.reward		100%		n/a	0	6	0	21	0	6	0	2
com.bds.order.domain.reward		100%		100%	0	6	0	7	0	4	0	2
com.bds.order.infrastructure.orderReward		100%		n/a	0	6	0	12	0	6	0	4
com.bds.order.domain.orderReward		100%		n/a	0	2	0	2	0	2	0	1
com.bds.order.infrastructure.config		100%		n/a	0	3	0	7	0	3	0	2
com.bds.order.infrastructure.common		100%		n/a	0	1	0	1	0	1	0	1
Total	20 of 3,269	99%	1 of 106	99%	4	224	3	625	3	169	0	52

목표 커버리지 Line, Branch 70% 달성 (total 244)

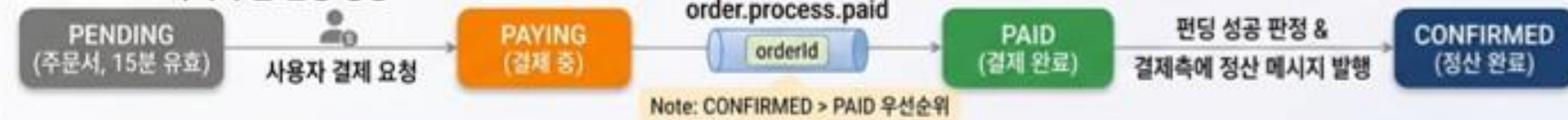
Funding 생명주기 및 메시지 발신 흐름



Order 상태 머신 및 메시지 수신 흐름

Order State Machine

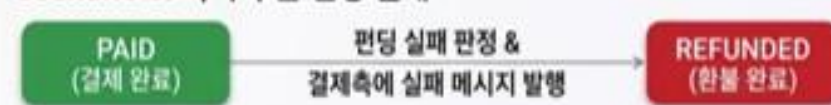
Scenario 1: 즉시 주문 편딩 성공



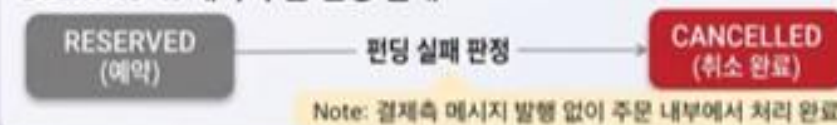
Scenario 2: 예약 주문 편딩 성공



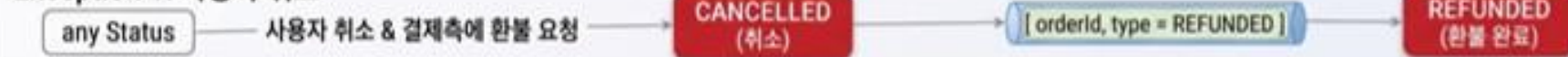
Scenario 3: 즉시 주문 편딩 실패



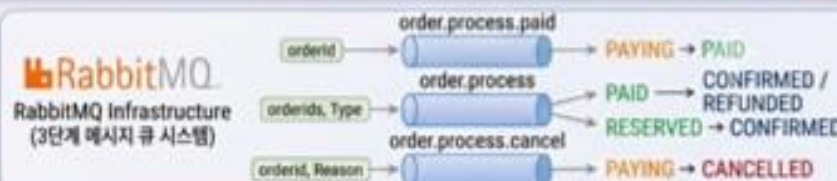
Scenario 4: 예약 주문 편딩 실패



Exception 1: 사용자 취소



Exception 2: 결제 실패





SERVICE 04

결제

결제 서비스 설계와 Transactional Outbox 기반 이벤트 처리 구조를
다룹니다.

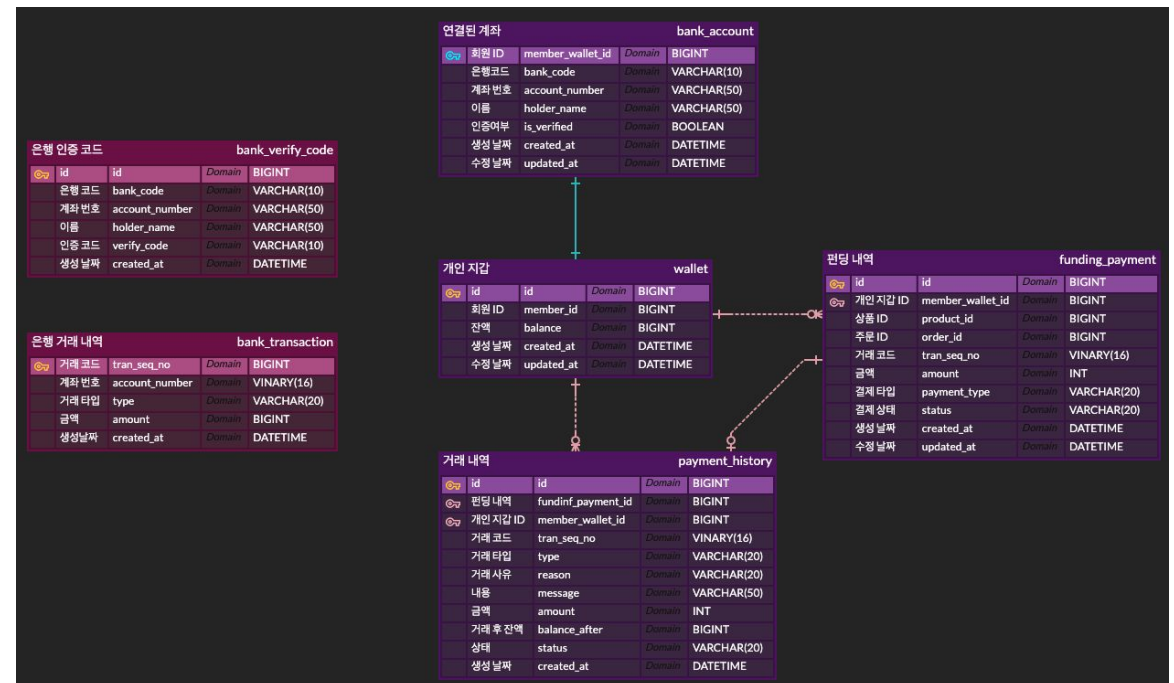
ERD

가상금융망 (Mock)

- 계좌연동을 위한 은행망
- 페이 충전, 환전을 담당

결제 서비스

- 자체 페이 서비스, 인당 1개의 계좌
- 주문 도메인에서 진행되는 펀딩 결제 담당



EndPoint

payment 도메인 엔드포인트 목록

method	path	auth required	설명
GET	/api/payment/wallet	O	월렛 잔액 조회
POST	/api/payment/accounts	O	계좌 등록
POST	/api/payment/accounts/verify	O	1원 인증
POST	/api/payment/deposit	O	페이 충전
POST	/api/payment/withdraw	O	페이 출금
GET	/api/payment/history	O	개인 월렛 거래 내역 조회
POST	/api/payment/funding	X	펀딩 결제 (RabbitMQ 로 구현 예정)
POST	/api/payment/funding/settlement	X	펀딩 정산 요청 (RabbitMQ 로 구현 예정)
POST	/api/payment/funding/batch-refund	X	펀딩 배치 환불 요청 (RabbitMQ 로 구현 예정)
POST	/api/payment/refund	O	수동 환불 (RabbitMQ 로 구현 예정)

가상 금융망 엔드포인트 목록

method	path	auth required	설명
POST	/api/banks/accounts	X	계좌 실명 조회 + 1원 송금
POST	/api/banks/accounts/verify	X	인증 코드 조회
POST	/api/banks/withdraw	X	계좌 출금 (페이 충전)
POST	/api/banks/deposit	X	계좌 입금 (원화 환불)
GET	/api/banks/transactions/{tranSeqNo}	X	거래 코드 조회

Jacoco Coverage

payment-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.bds.payment.bank.application		100%		100%	0 12	0 22	0 9	0 1
com.bds.payment.bank.domain.bankTransaction		100%		n/a	0 1	0 7	0 1	0 1
com.bds.payment.bank.domain.bankVerifyCode		100%		n/a	0 1	0 8	0 1	0 1
com.bds.payment.bank.domain.common		100%		n/a	0 1	0 2	0 1	0 1
com.bds.payment.bank.infrastructure.persistence.BankTransaction		100%		n/a	0 5	0 19	0 5	0 2
com.bds.payment.bank.infrastructure.persistence.bankVerifyCode		100%		n/a	0 6	0 21	0 6	0 2
com.bds.payment.bank.presentation.controller		0%		n/a	5 5	5 5	5 5	1 1
com.bds.payment.bank.presentation.request		100%		n/a	0 3	0 3	0 3	0 3
com.bds.payment.bank.presentation.response		100%		n/a	0 6	0 6	0 6	0 4
com.bds.payment.payment.application.accounts		100%		100%	0 8	0 23	0 5	0 1
com.bds.payment.payment.application.funding		100%		100%	0 10	0 22	0 5	0 1
com.bds.payment.payment.application.payment		93%		100%	1 11	1 33	1 8	0 2
com.bds.payment.payment.application.wallet		100%		100%	0 15	0 27	0 13	0 1
com.bds.payment.payment.domain.account		100%		n/a	0 3	0 19	0 3	0 1
com.bds.payment.payment.domain.common		100%		n/a	0 5	0 10	0 5	0 5
com.bds.payment.payment.domain.fundingPayment		100%		n/a	0 2	0 11	0 2	0 1
com.bds.payment.payment.domain.paymentHistory		100%		n/a	0 1	0 11	0 1	0 1
com.bds.payment.payment.domain.wallet		100%		100%	0 4	0 12	0 3	0 1
com.bds.payment.payment.global.exception		78%		0%	9 13	25 67	7 11	0 3
com.bds.payment.payment.infrastructure.config		100%		n/a	0 2	0 9	0 2	0 1
com.bds.payment.payment.infrastructure.external		3%		0%	5 6	38 39	4 5	0 1
com.bds.payment.payment.infrastructure.external.request		100%		n/a	0 6	0 6	0 6	0 3
com.bds.payment.payment.infrastructure.external.response		28%		n/a	3 4	3 4	3 4	2 3
com.bds.payment.payment.infrastructure.persistence.account		91%		100%	2 11	3 29	2 8	0 3
com.bds.payment.payment.infrastructure.persistence.fundingPayment		96%		n/a	1 8	2 33	1 8	0 3
com.bds.payment.payment.infrastructure.persistence.paymentHistory		81%		n/a	2 7	2 32	2 7	0 3
com.bds.payment.payment.infrastructure.persistence.wallet		91%		83%	2 12	1 26	1 9	0 3
com.bds.payment.payment.presentation.controller		0%		n/a	8 8	8 8	8 8	4 4
com.bds.payment.payment.presentation.request		73%		n/a	1 5	1 5	1 5	1 5
com.bds.payment.payment.presentation.response		54%		n/a	6 15	13 31	6 15	3 8
Total	469 of 2,607	82%	6 of 51	88%	45 196	102 550	41 170	11 70

각 서비스 기준 커버리지 100%

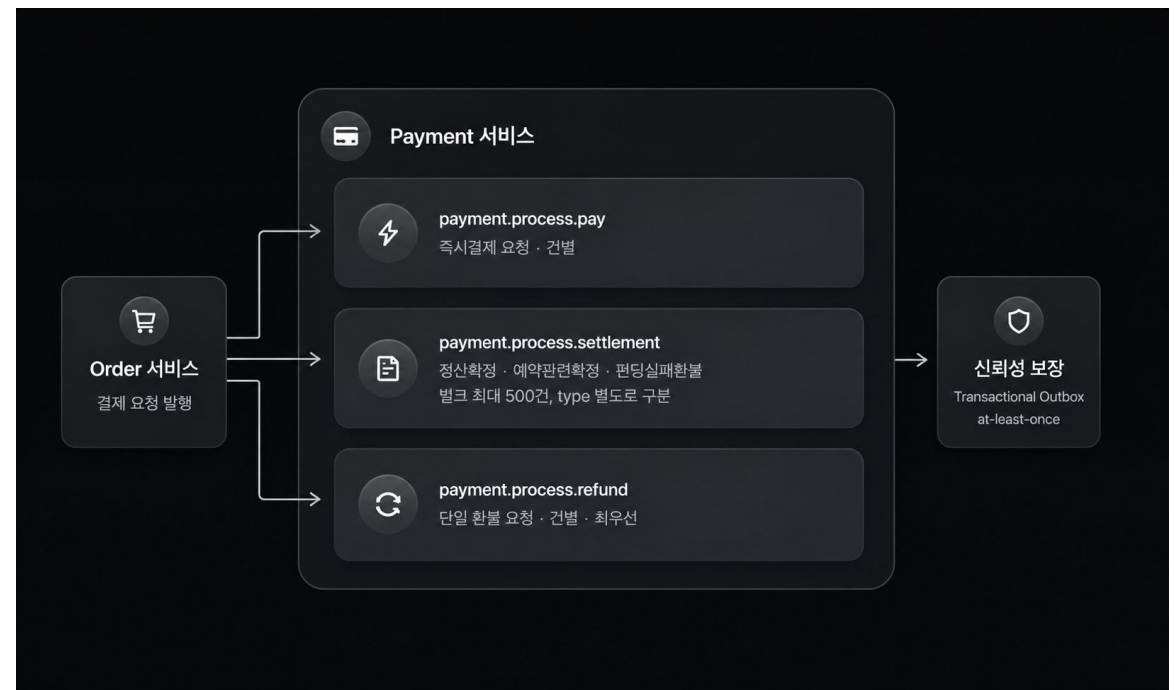
결제 흐름

설계 목적

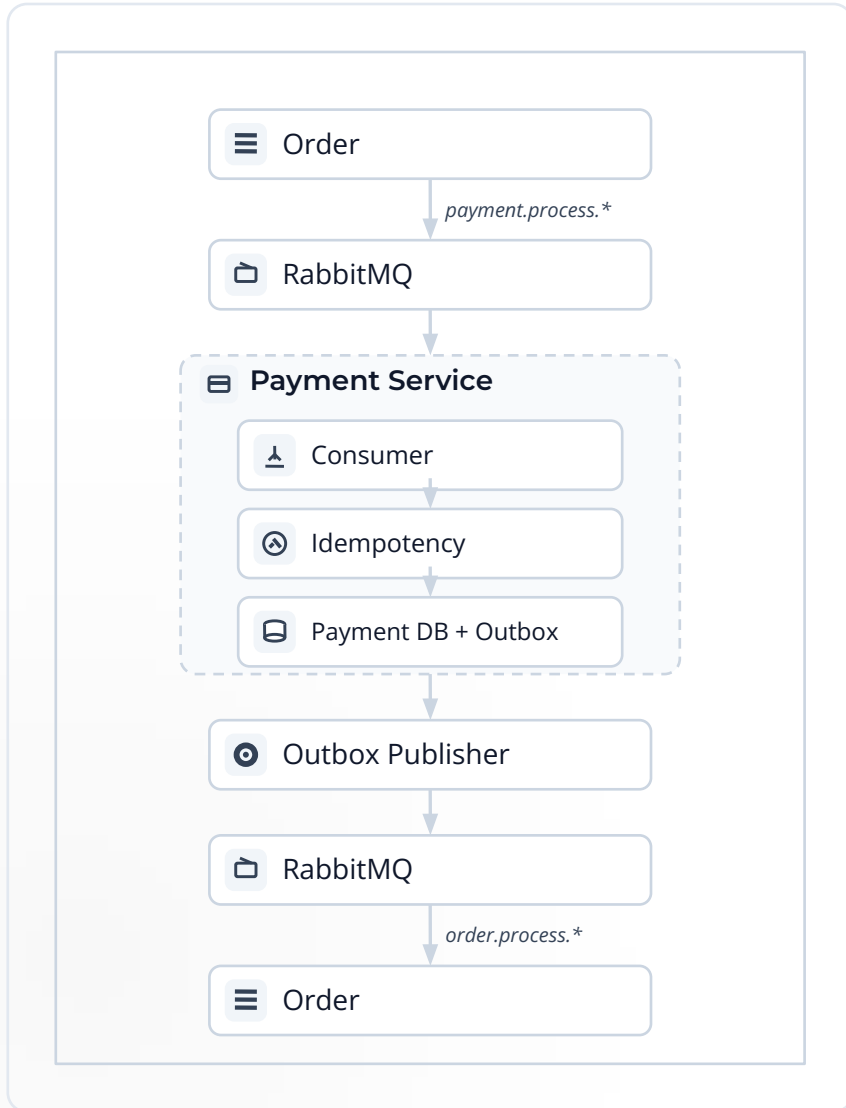
- Payment 상태와 이벤트의 일관성 보장
- 메시지 유실 방지
- 안정적인 비동기 이벤트 처리

적용 기술

- Transactional Outbox
- Idempotency (requestId, batchId)
- RabbitMQ (at-least-once)



결제 설계 핵심



- Payment 처리와 Outbox를 동일 트랜잭션으로 저장합니다.
- Commit 이후 Publisher가 이벤트를 발행합니다.
- 이벤트 재전송을 고려한 멍등성을 보장합니다.
- Order 서비스는 결과 이벤트를 받아 상태를 변경합니다.



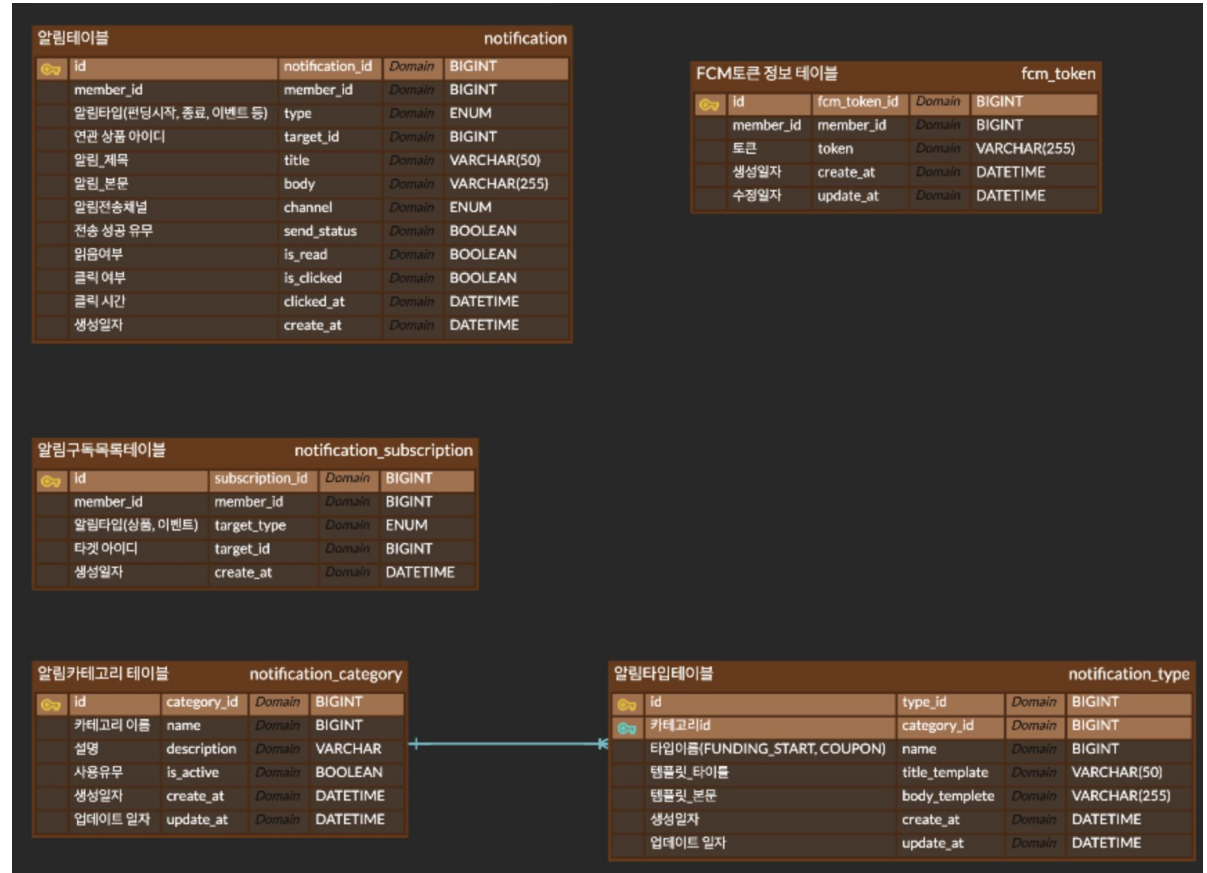
SERVICE 05

알림

알림 도메인 설계와 공용 알림 테이블, SSE / FCM Fallback 전송 구조를
다룹니다.

ERD

- FUNDING 상품이나 프로모션 이벤트의 알림 구독 정보를 저장할 Notification Subscription 테이블
- Funding, Order 등의 상태에 따라 생성되는 Notification 테이블
- 이후 외부 알림을 위한 FCM 토큰 저장 테이블
- 관리자 알림 기능을 위한 알림 타입, 알림 카테고리 테이블로 구성



EndPoint

method	path	auth required	설명
GET	/api/notifications/connect	O	SSE 연결 (알림 구독)
GET	/api/notifications	O	알림 목록 조회 + 전체 읽음 처리
GET	/api/notifications/unread-count	O	읽지 않은 알림 수 조회
POST	/api/notifications/subscriptions	O	알림 구독 등록
DELETE	/api/notifications/subscriptions	O	알림 구독 해지
POST	/api/notifications/fcm-token	O	FCM 토큰 저장
DELETE	/api/notifications/fcm-token	O	FCM 토큰 삭제

Jacoco Coverage

NotificationService

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● connect(Long)		100%		n/a	0	1	0	10	0	1
● createFundingNotification(FundingNotificationCommandDto)		83%		75%	2	7	2	15	0	1
● createNotification(Long, NotificationType, String, String, String, NotificationChannel)		100%		n/a	0	1	0	4	0	1
● createOrderNotification(OrderNotificationMessageDto)		89%		83%	1	5	1	10	0	1
● getNotifications(Long, Pageable)		100%		n/a	0	1	0	7	0	1
● getUnreadCount(Long)		100%		n/a	0	1	0	2	0	1
● lambda\$createFundingNotification\$0(FundingNotificationCommandDto, String, String, Long)		100%		n/a	0	1	0	2	0	1
● lambda\$unsubscribe\$0()		100%		n/a	0	1	0	1	0	1
● subscribe(Long, SubscriptionTargetType, Long)		83%		100%	0	2	2	8	0	1
● unsubscribe(Long, SubscriptionTargetType, Long)		100%		n/a	0	1	0	5	0	1
Total	21 of 265	92%	3 of 16	81%	3	21	5	63	0	10

Notification Service 위주로 테스트 커버리지를 확대했으며, 전체 코드 커버리지는 80% 이상입니다.

공용 알림 테이블 설계

type + targetId 기반 통합 알림 테이블

타입이 늘어나도 테이블은 하나 — 랜딩 URL 결정은 클라이언트에 위임

BEFORE — 타입별 테이블 분리

funding_notifications

order_notifications

promotion_notifications

event_notifications ...?

chat_notifications ...?



타입 추가마다
테이블 증가

AFTER — notifications 테이블 하나로 통합

type	targetId (VARCHAR)	클라이언트 랜딩
FUNDING_START	"123"	/products/123
PAID	"order-1234-5678"	/orders/order-1234-5678
PROMOTION	"event-99"	/events/event-99

✓ 새 타입 추가 시 테이블 변경 없이 type 값 + 프론트 라우팅 규칙만 추가

type + targetId(String) 조합으로 모든 알림을 하나의 테이블에서 관리 → 확장에 열린 구조

알림 - SSE / FCM Fallback 알림 전송 구조

사이트 접속 중이면 **SSE Toast**, 접속 중이 아니면 **FCM** 푸시로 자동 전환

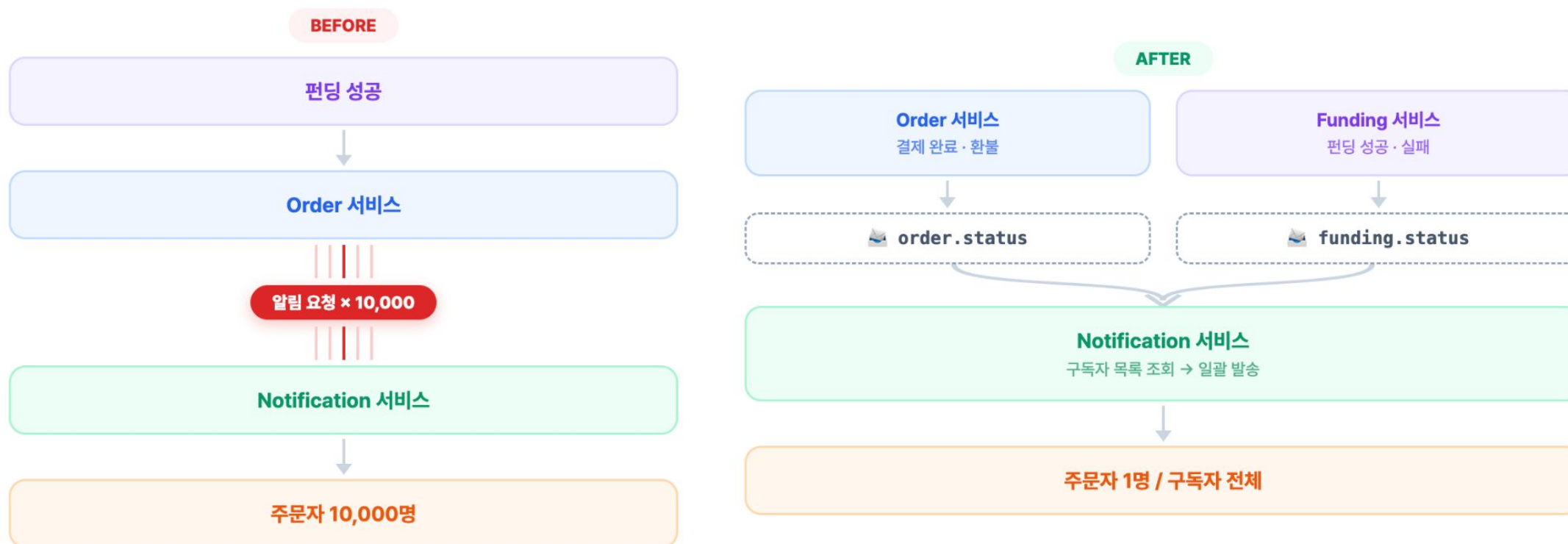
사용자가 사이트 밖에 있어도 편당·결제 등 중요한 알림을 놓치지 않도록 SSE → FCM 이중 채널로 구성



사용자가 사이트 밖에 있어도 편당·결제 등 중요한 알림을 놓치지 않도록 SSE → FCM 이중 채널로 구성

RabbitMQ Order / Funding 큐 분리

이벤트는 1번만 발행하고, 구독자 조회·일괄 발송은 Notification이 전담



이벤트 발행은 서비스당 1번 — 구독자 수는 Order·Funding이 알 필요 없이 **Notification**이 **notification_subscription**으로 **일괄 처리** → 부하 분산 + 책임 분리



질문 및 답변

궁금하신 점을 자유롭게 질문해 주세요



감사합니다

끝까지 함께해 주셔서 감사합니다